# Deep double descent (notes)

Jake Mendel Kaarel Hänni

April 2023

## 1 Questions

### 1.1 Which patterns are learned quicker?

- Why are memorised patterns learned quicker?
- A few ways to train toy LMs in more realistic scenarios:
  - There are two tasks. num+num mod p, and letter + letter mod q. It has to learn both. Do we see two groks or dds? This can also be done by having a special initial token to differentiate the tasks.
  - How does one LM learning two tasks compare to learning each of them separately? Do the tasks decompose? Does this depend on how similar the tasks are? eg. if one is addition mod p and the other is addition mod q does it learn two generalising circuits, or one and memorises p and q.
  - Can we come up with a toy example of a single large epoch? eg. p is large, so we give lots ($\sim p^2$) of datapoints but only run one epoch. What do we observe in this scenario
  - Is there a rule governing which group operations are learned faster? (Both when trained in parallel, and when trained separately. Are "pattern complexities" according to these two metrics correlated?)

### 1.2 Testing Hypotheses about Memorization

- Can we keep going with the epsilon orthogonal idea to work out how to memorise $d^2$ datapoints per layer not d? Or more generally, come up with any constructive way to write a memorising circuit which memorises one datapoint per parameter. Can we test empirically whether this happens?
- Where is the interpolation threshold? For a fixed interpolation threshold (defined as the point when train error drops below some $\epsilon$) we expect something qualitatively like

$$P = P_{\text{min}} + \frac{1}{E - E_{\text{min}}} \tag{1}$$

  where $P$ is the number of parameters, $E$ is the number of Epochs, $P_{\text{min}}$ is the minimum number of parameters required in the infinite epochs limit, and $E_{\text{min}}$ is the minimum number of Epochs in the infinite parameter limit. We think that $P_{\text{min}} \sim n_{\text{data}}$ (the number of data points).
  - In the original DDD paper, can we work out the actual number of parameters they are using, not just the residual dimension?
  - How does $P_{\text{min}}$ scale with the number of labels? Information theory suggests it might be like $n_{\text{data}} \log k$.

- At/before interpolation threshold, how changed are the input vectors of test data by the network?
- Understand if past the interpolation threshold, it's the memorising circuit or generalising circuit that dominates predictions with train data. eg. early in training and late in training, is it the same parameters (or some equivalent set??) that classifies a datapoint from the training set? Does this depend on regularization?
- What's going on with randomization exaggerating DDD? It makes sense because more randomization privileges that memorization circuits over generalisation. With regularisation, if generalising circuit requires more weights than any memorization circuit, there is some fraction of randomized labels that prohibit generalising circuit from ever forming.
- Can we study the creation of the generalising circuit?
  - Can we train a simple classifier on internal activations of a post-DDD model trained on data with some random labels to determine if a datapoint had a random label or not. Does this classifier become much better around the interpolation threshold? If this is true, then the model is using generalising circuit when it can, and memorization circuit otherwise. If not, then it is just using memorization always, and generalising circuit is kinda coming along for the ride.
  - Can we get rid of generalising circuit, destroying test performance, without destroying train performance?
- Is there some other way to detect data points a model has memorized? (see `https://transformer-circuits.pub/2023/toy-double-descent/index.html#comment-mnist`)

### 1.3 Misc

- Does grokking require regularisation? Find Bilal's paper
- Which of these phenomena work better epoch wise, and which modelwise? Can we get better at switching between them? Are they actually different phenomena?
- Is there a connection between Chinchilla scaling laws and effective model complexity?
- Note that grokking and DD may well not occur without anything equivalent to regularisation (no Adam). We had an argument that grokking implies L2 complexity of generalising circuit ¿ memorizing circuits therefore gradient updates from a whole epoch shared across all memorizing circuits is less than across generalising circuit, so G catches up to M. The reason why the argument fails is that if M is learned first, then we may have a scenario whereby correct logit gets +100 from M and +1 from G. Due to nonlinearity this means that G would get a tiny update and M a big one. Therefore if $\mathscr{C}^{L2}(M)/\mathscr{C}^{L2}(G)$ is greater than the ratio of the gradient updates, M will stay ahead of G.

### 1.4 The relevance of deep double descent to LLMs (and other SOTA models)

- We are not across the interpolation threshold, but there might be smaller interpolation thresholds for lots of smaller tasks and we have passed each of these thresholds.
- What can we say about training time double descent when it's all for one epoch, but that epoch is bloody long? Can we make any predictions about how performance would change on a second epoch?

- Are there any tasks (what are they) for which we could observe modelwise/epochwise grokking in LM/image classifier training? Possible options include
  - Addition (though even GPT-4 struggles with that)
  - Indirect object identification
  - Image classifier grokking
- Take EleutherAI's Pythia suite (various training checkpoints). Does it grok/double descent addition? When? how many parameters are involved in each subtask? Does this act like a smaller model trained on only that task?
- Train a language model for many epochs. Does it DD? (+ the same question for model size)

# 2 Links/comments

## 2.1 Progress Measures for Grokking via Mechanistic Interpretability

It says that memorizing components are forgotten after the generalizing circuit is formed, but says this is due to regularization; unclear if this would happen without regularization.

"In the previous section, we saw that each phase of grokking corresponded to an inflection point in the $\ell_2$-norm of the weights. This suggests that weight decay is an important component of grokking and drives progress towards the generalizing solution. In Appendix D.1, we provide additional evidence that weight decay is necessary for grokking: smaller amounts of weight decay causes the network to take significantly longer to grok (echoing the results on toy models from Liu et al. (2022)), and our networks do not grok on the modular arithmetic task without weight decay or some other form of regularization."

## 2.2 Double descent theoretical model

starting page 22 here: `http://people.csail.mit.edu/moitra/docs/408lec6post.pdf`
continuing here: `http://people.csail.mit.edu/moitra/docs/408lec7post.pdf`

## 2.3 https://arxiv.org/pdf/2103.03404.pdf

paper saying that transformers are ensembles of shallow models

## 2.4 Slingshot mechanism

`https://arxiv.org/pdf/2206.04817.pdf` claims a connection between grokking and the model doing some weird cyclic thing during the phase of training with perfect train accuracy

I buy that they show that these both show up in the same parameter regime, but from a skim I was not able to understand what empirical evidence they have for a causal connection between the two beyond this. I think they say that grokking happens during the onset of something having to do with the slingshot mechanism in text, but I have not understood if this is concretely referring to one of their experiments or not

## 2.5 Misc Notes

Anthropic paper on memorisation. Toy models of superposition also Generalising circuits massive gini coefficient? Look at a paper that cites Neel's

Effective model complexity is defined for a training regimen (eg model size and number of epochs specified) by: $\mathcal{C}_\epsilon$ is the maximum number of independent data points that can be in the training set for train error to reach below $\epsilon$ by the end of this training regimen, on expectation.

## 3 Constructing a memorizing NN by hand

task setup: $m$ input vectors in $\mathbb{R}^n$ with entries that are independent random $\pm 1$ coinflips, labels are also independent random $\pm 1$ (call the corresponding vectors red vs blue)

architecture: neural net which has residual stream $= \mathbb{R}^n$ to which $Relu(Wx)$ is added in each layer, where $W \in \mathbb{R}^{n \times n}$ (like a resnet but not quite because the residual stream can contain negative stuff; I do not see any reason why this is crucial though — the same argument should work with vectors that have $\pm 1$ entries replaced by $\{0, 1\}$, keeping everything non-negative)

We will process inputs into the format where one coordinate being $> \log n$ corresponds to the input being red, changing only one coordinate for each red vector and changing nothing for blue vectors. (This processing is the difficult step. Once we have this, it can easily be turned into output logits as follows with 2 more layers. The first one picks out large coordinates and amplifies them to order $n^3$, and the second sums coordinates minus bias $n^2$. This will be large for red vectors, small for blue vectors.) Each layer will pick a random set of $n^2/\log m$ red vectors and leave them all with exactly one coordinate $> \log n$. In fact, we construct each row of $W$ by picking a random set of $n/\log m$ red input vectors, finding a common direction which these exactly max out among the set of all inputs (and the output format implies also among the processed inputs), and having a proper scaling applied to get the output to have order $\log n$ for each. Such a direction can be found just by averaging the vectors (a probabilistic argument shows that these vectors are indeed maximally far in this direction, I think).